AMATH 563: COMPUTATIONAL REPORT

CAMILLE BERGERON

Economics Department, University of Washington, Seattle, WA
cghb@uw.edu

1. Introduction

We consider the Lotka-Volterra predator-prey system, a classical system of ODEs governing population dynamics. Rather than assuming knowledge of the governing equations, we treat the system as a black box and attempt to learn its dynamics purely from observations of the state trajectory. Specifically, given 50 observations of the prey and predator populations at times $t_n = 0.4n$, $n = 0, \ldots, 49$, we design a two-stage kernel regression pipeline to recover the unknown vector field $f = (f_1, f_2)$ driving the system. In the first stage, we smooth the observed trajectories using an RBF kernel and differentiate analytically to obtain derivative estimates. In the second stage, we learn the map from state space to derivatives using a second kernel Γ, comparing three kernel families: RBF, polynomial, and exponential. We find that the degree-2 polynomial kernel achieves the lowest error across all metrics, with a mean absolute error of 0.1564 for f_1 and 0.1232 for f_2.

2. Theory

We design a kernel method to learn $\hat{f}_i \approx f_i(p_1, p_2) = \frac{dp_i}{dt}$ from observations $\mathbb{R}^2 \ni y(t_n) = (p_1(t_n), p_2(t_n))$ given $n = 0, \ldots, 49$ and $t_n = 0.4n$.

We seek to solve

$$f^* \approx \hat{f} = \operatorname{argmin}_{f \in \mathcal{H}} \frac{1}{2}\|f(X) - y\|_2^2 + \frac{\lambda^2}{2}\|f\|_{\mathcal{H}}^2, \tag{1}$$

where $f(X) = (f(x_1), \ldots, f(x_n)) \in \mathbb{R}^n$, f^* is the true function value, $y \in \mathbb{R}^m$, $A \in \mathbb{R}^{m \times n}$, and $\mathcal{H}$ is the RKHS associated with a PDS kernel K. A closed form solution of (1) is

$$f^* = \sum_{i=1}^{n} \alpha_1 K(x, x_i) = K(x, X)\left((K(X, X) + \lambda_K I)^{-1} y\right) \tag{2}$$

. with $K(x, x)$ as our chosen kernel. By differentiating (2), we can analytically estimate the state derivatives (e.g. velocities) at the mesh points $y(t_n)$. We can do this because $K(X, X)$ is known (such as a Gaussian/RBF), so its derivatives with respect to its first argument x are also known.

These derivatives are

$$\partial^{\alpha^j} f^{(i)}(x) = \partial^{\alpha^j} K(x, X)\left(K(X, X) + \lambda_K I\right)^{-1} f \tag{3}$$

With this, we can then learn the true functional form f^* to learn the map P that relates the state variables i.e. find $P(s) = f$, where s is our state variables. To do this, we learn

$$P(s) = \Gamma(s, S)\left((\Gamma(S, S) + \lambda_\Gamma I)^{-1} f\right) \tag{4}$$

where $\Gamma(S, S)$ is a another kernel.

Date: May 19, 2026.

3. Methods

We first want to estimate (1). Note that we solve (1) twice (once to obtain $\hat{f}_1$ and once to obtain $\hat{f}_2$) treating each component regression independently. Choosing a polynomial and rbf basis as $K(X, X)$. Our inputs are

$$X = \begin{bmatrix} p_1(t_0) & p_2(t_0) \\ p_1(t_1) & p_2(t_1) \\ \vdots & \vdots \\ p_1(t_{49}) & p_2(t_{49}) \end{bmatrix} \in \mathbb{R}^{50\times 2}$$

with targets

$$y_1 = \begin{bmatrix} \dot{p}_1(t_0) \\ \vdots \\ \dot{p}_1(t_{49}) \end{bmatrix}, \quad y_2 = \begin{bmatrix} \dot{p}_2(t_0) \\ \vdots \\ \dot{p}_2(t_{49}) \end{bmatrix}$$

. We estimate the derivatives of X using gradients and then fit y_1 and y_2 using our first model choice.

Then, with these models we take the new derivative matrix and estimate another model fit with a new kernel to get the correct mapping.

For $K(X, X)$ an rbf kernal was chosen with parameters $\lambda = 1e^{-}3$ and $\gamma = 1$.

For $\Gamma(S, S)$, the following kernels were (naively) considered:

Model	**Kernel**	**Hyperparameters**	λ
`rbf_lam0.2`	RBF	$\gamma = 1.0$	0.2
`rbf_lam1.0`	RBF	$\gamma = 1.0$	1.0
`rbf_lam6.0`	RBF	$\gamma = 1.0$	6.0
`rbf_gam0.1`	RBF	$\gamma = 0.1$	0.001
`rbf_gam1.0`	RBF	$\gamma = 1.0$	0.001
`rbf_gam10.0`	RBF	$\gamma = 10.0$	0.001
`poly_d2`	Polynomial	$d = 2,\ c_0 = 1$	0.001
`poly_d3`	Polynomial	$d = 3,\ c_0 = 1$	0.001
`poly_d4`	Polynomial	$d = 4,\ c_0 = 1$	0.001
`exp_sig0.5`	Exponential	$\sigma = 0.5$	0.001
`exp_sig1.0`	Exponential	$\sigma = 1.0$	0.001
`exp_sig5.0`	Exponential	$\sigma = 5.0$	0.001

Table 1. Summary of all models tested for $\Gamma(S, S)$. RBF kernel: $e^{-\gamma\|s-s'\|^2}$; Polynomial kernel: $(\langle s, s'\rangle + c_0)^d$; Exponential kernel: $e^{-\|s-s'\|/\sigma}$. λ is the nugget regularisation parameter.

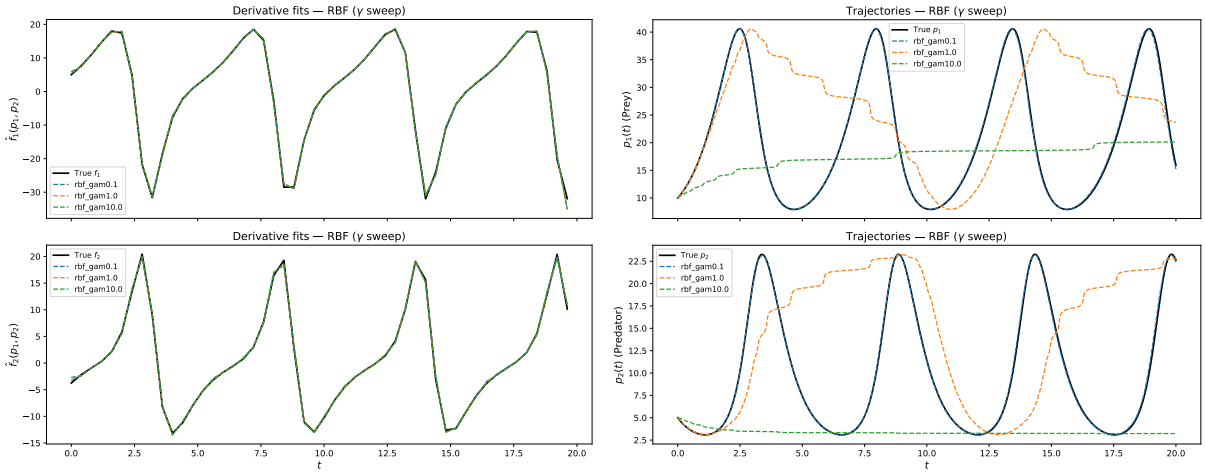
4. Results

Looking at the derivatives for the model comparisons, we have pretty okay fits for the various models with varying degrees. For $\Gamma(S, S)$ as rbf, we have really good fits for f but varying degrees of fit for p_i. Evidently, `rbf_gam0.1` performs the best of this class, as seen in Figure 1.

For polynomial functions, most do pretty well on f but very different performances on p_i, with `poly_d2` doing the best from initial analysis as seen in Figure 2.

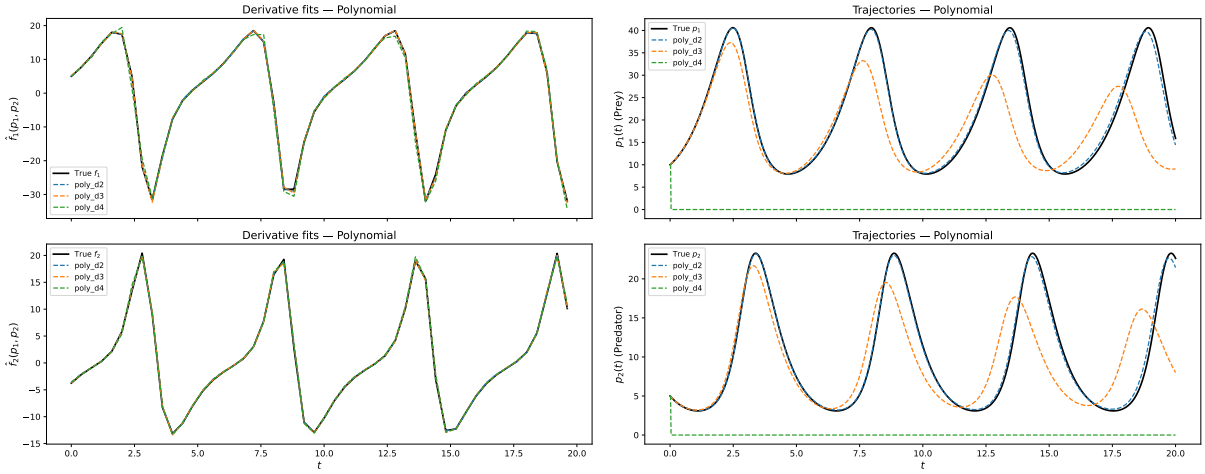
Exponential has similar success in f but its best match to p_i is `exp_sig5.0`.

As a measure for fit from $\hat{f}_i$ to f^*, I chose to look at maximum absolute error i.e. $\max \|\hat{f} - f^*\|$ and the average over all sampled point $\frac{1}{N}\sum_N |\hat{f} - f^*|$ where $N = 50$. The results are in Table 2



(A) Model Comparison (B) Trajectory Comparisons

FIGURE 1. Comparison of learned models and simulated trajectories for RBF



(A) Model Comparison (B) Trajectory Comparisons

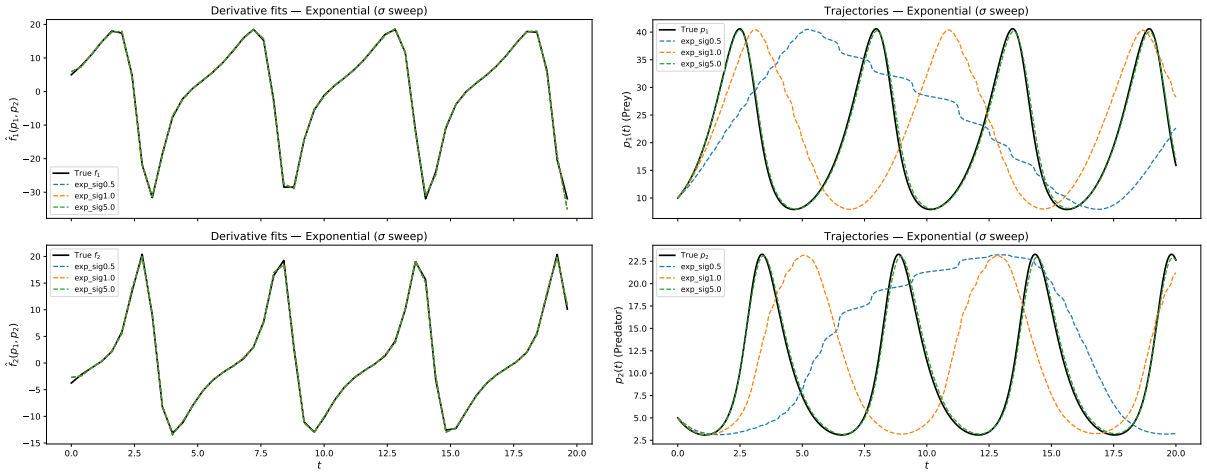
FIGURE 2. Comparison of learned models and simulated trajectories for Polynomial

As we saw earlier, `poly_d2` achieves the lowest error in all cases (for both functions and from both metrics).

5. Summary and Conclusions

In this report, we designed and implemented a two-stage kernel regression method to learn the unknown vector field of the Lotka-Volterra system from sparse observations. In the first stage, an RBF kernel smoother was fit on the time domain to obtain analytic derivative estimates of the observed trajectories. In the second stage, we learned the map $P(s) = \Gamma(s, S)(\Gamma(S, S) + \lambda_\Gamma I)^{-1} f$ using three families of kernels for Γ: RBF, polynomial, and exponential.

Across all models, the derivative fits $\hat{f}_i$ were reasonably accurate, but trajectory simulation revealed more significant differences between kernel choices. The degree-2 polynomial kernel `poly_d2` achieved the best performance overall, with the lowest mean and maximum absolute errors for both $\hat{f}_1$ and $\hat{f}_2$. This is perhaps unsurprising given that the true Lotka-Volterra dynamics are themselves



(A) Model Comparison (B) Trajectory Comparisons

FIGURE 3. Comparison of learned models and simulated trajectories for Polynomial

Model	**Mean** f_1	**Mean** f_2	**Max** f_1	**Max** f_2
`rbf_lam0.2`	1.5988	0.9700	6.1552	3.8712
`rbf_lam1.0`	5.2030	3.0869	16.4805	10.0299
`rbf_lam6.0`	10.0135	5.8559	27.5434	16.6284
`rbf_gam0.1`	0.3595	0.2389	3.1125	0.8057
`rbf_gam1.0`	0.3639	0.2374	3.1649	0.9568
`rbf_gam10.0`	0.3672	0.2374	3.1563	1.0714
`poly_d2`	0.1564	0.1232	0.7660	0.4319
`poly_d3`	0.2569	0.1691	1.2209	0.7844
`poly_d4`	0.7359	0.2223	3.7865	1.1024
`exp_sig0.5`	0.3670	0.2374	3.1623	1.0724
`exp_sig1.0`	0.3669	0.2375	3.1653	1.0695
`exp_sig5.0`	0.3655	0.2373	3.1646	1.0508

TABLE 2. Mean and maximum absolute errors for $\hat{f}_1$ and $\hat{f}_2$ across all kernel models. RBF models are shown with varying regularisation λ and bandwidth γ, polynomial models with varying degree, and exponential (Laplace) models with varying bandwidth σ.

polynomial (quadratic) in the state variables, making the degree-2 polynomial kernel a natural fit. Among RBF models, varying the bandwidth γ had more impact than varying the regularization λ, while exponential kernels performed comparably to RBF across all bandwidth choices. A more rigorous methodology for choosing the hyperparameter could be explored and model selection in general could be worked on to obtain a better estimat $\hat{f}$.

ACKNOWLEDGEMENTS

The author is thankful to Professor Hosseini for useful discussions about regularization and for providing great lecture notes, as well as for allowing me an extension on this assignment. Also thank you to TA Juan for being so responsive and understanding!